

INGENIERÍA DE SOFTWARE Y GESTIÓN DE DATOS

Procedimiento de Mantenimiento de Código y Preservación de Estructuras Vivas en Producción y Capacitación

En sistemas de misión crítica de nivel industrial como nuestro **ERP/CRM Automotriz**, la modificación del código fuente y el mantenimiento evolutivo presentan un desafío fundamental: **¿cómo alterar las reglas del negocio y las estructuras de la base de datos sin destruir la información operativa ya existente?**

Este documento establece formalmente el procedimiento de ingeniería, control de versiones de bases de datos y segregación de ambientes para garantizar un impacto de riesgo cero sobre los datos durante el pase a producción y los ciclos de ajuste solicitados por los usuarios en fase de capacitación.

1. La Regla de Oro: Control de Versiones de Base de Datos (Migraciones)

El error más común y catastrófico en el mantenimiento de software es modificar la base de datos de producción de forma manual (ej. ingresando al dashboard web de Supabase y agregando una columna con clics o ejecutando un script suelto). Esto destruye la trazabilidad e impide replicar el entorno de manera segura.

Para mitigar esto, implementamos el enfoque **Migration-Based Database Development** mediante **Supabase CLI**:

- **El código y el esquema van juntos:** Cada cambio estructural en las tablas de la base de datos debe ser representado por un archivo de migración incremental en lenguaje SQL (un archivo con una marca de tiempo secuencial, ej. `20260619143000_add_kilometraje_ot.sql`).
- **Los archivos son inmutables:** Una vez que una migración es empujada a un servidor, ese archivo jamás vuelve a modificarse. Si el usuario solicita un ajuste posterior, se genera una nueva migración que corrige o complementa la anterior.

2. Flujo de Trabajo Técnico ante una Solicitud de Ajuste del Usuario

Cuando un usuario está en periodo de capacitación y solicita un ajuste (ejemplo: *"Necesitamos que al dar de alta una Orden de Trabajo en el ERP, se capture obligatoriamente el nivel de combustible en el tanque con un rango del 0% al 100%"*), el procedimiento estricto es el siguiente:

Paso A: Desarrollo Local y Generación del Diff

El programador arranca su entorno de pruebas local aislado de Supabase y realiza el cambio en la interfaz o código del backend. Posteriormente, ejecuta la herramienta de Supabase CLI para generar automáticamente la migración estructural:

```
# El desarrollador modifica el esquema localmente y luego genera el archivo de migración
supabase db diff -f add_nivel_combustible_ot
```

Esto creará un archivo SQL en la carpeta `/supabase/migrations/` que contendrá de manera limpia sentencias puras como

```
ALTER TABLE ope_ordenes_trabajo ADD COLUMN nivel_combustible INT CHECK (nivel_combustible BETWEEN 0 AND 100);.
```

Paso B: Despliegue en el Ambiente de Capacitación (CI/CD)

El programador sube el código al repositorio (Git). El pipeline de Integración Continua (GitHub Actions/GitLab CI) intercepta el cambio y aplica únicamente las migraciones nuevas mediante la instrucción:

```
# El pipeline aplica de forma segura las migraciones no ejecutadas, sin tocar los datos existentes
supabase db push
```

¿Por qué no se pierden los datos aquí? Porque el comando `supabase db push` lee una tabla interna del servidor llamada `schema_migrations`. El motor detecta exactamente cuáles archivos SQL ya se corrieron en el pasado y cuáles son nuevos, ejecutando exclusivamente las líneas de código añadidas. Los registros de autos, clientes y turnos cargados por los usuarios durante su capacitación permanecen intactos.

COMANDO PROHIBIDO EN ENTORNOS VIVOS

Nunca, bajo ninguna circunstancia, se debe ejecutar el comando `supabase db reset` en un ambiente de Capacitación Activa o Producción. Dicho comando destruye por completo el esquema y las tablas para volver a crearlas desde cero, provocando una pérdida total e irreversible de los datos operativos. El pipeline debe estar bloqueado para no usar este parámetro fuera de Local.

3. El Patrón "Expandir y Contraer" (Expand and Contract Pattern)

Cuando el sistema ya se encuentra en **Producción Completa** con usuarios reales facturando y operando en el taller, cambiar la estructura de una tabla puede ser peligroso si el backend viejo intenta escribir en ella mientras se despliega el backend nuevo. Para garantizar un mantenimiento con cero pérdida de datos y cero tiempo de inactividad (Zero-Downtime), se utiliza este patrón de ingeniería:

Fase del Proceso	Acción en la Base de Datos (Estructura)	Acción en el Código Fuente (Backend/Frontend)	Estado de los Datos
1. Expandir (Fase Inicial)	Se agrega la nueva columna o tabla mediante migración incremental (<code>ALTER TABLE</code>). Si se requiere cambiar un tipo de dato, se crea una columna temporal.	El código nuevo es desplegado. Está programado para escribir tanto en la estructura vieja como en la nueva simultáneamente (Dual-Write).	Seguro. Ningún dato viejo se elimina. Los registros nuevos llenan ambos campos para no romper la compatibilidad.
2. Migrar (Transición)	La estructura permanece idéntica a la fase 1.	Se ejecuta un script en segundo plano (Data Migration Script) que copia los registros del formato viejo al formato nuevo exclusivamente para las filas históricas.	Seguro. Se unifica la información histórica con la nueva información recopilada en caliente.
3. Contraer (Fase Final)	Se genera una última migración que elimina la columna vieja o remueve las restricciones en desuso (<code>DROP COLUMN</code> viejo).	Se remueve la lógica de doble escritura del código. El backend ahora opera al 100% apuntando únicamente a la nueva estructura optimizada.	Seguro. La estructura vieja se retira solo cuando se certificó que el nuevo código es estable.

4. Segregación de Ambientes y Políticas de Datos

Para evitar accidentes lógicos y pérdidas de información, el proyecto debe operar bajo tres bases de datos de Supabase completamente independientes, gobernadas por políticas diferenciadas:

- Ambiente Local (Desarrollo):** Datos 100% volátiles y ficticios. El programador puede destruir y reconstruir la BD las veces que sea necesario mediante variables semilla (`seed.sql`).
- Ambiente de Staging / Capacitación:** Contiene datos reales ingresados por los usuarios para simular el día a día del taller. El código se actualiza continuamente mediante el pipeline CI/CD. Está prohibida la manipulación manual de registros por consola.
- Ambiente de Producción (Go-Live):** Blindaje absoluto. Acceso restringido únicamente mediante llaves encriptadas del pipeline. Copias de seguridad automáticas activas por hora (Point-in-Time Recovery).

5. El Protocolo de "Purga Operacional Limpia" antes del Go-Live

Un escenario muy común que preocupa a las empresas es: *"¿Cómo hacemos para borrar todas las órdenes de trabajo falsas, pruebas de facturación SIFEN y clientes de juguete que creamos durante la capacitación, pero conservar los repuestos que cargamos al inventario, las configuraciones de los mecánicos y los accesos de usuario para poder arrancar el lunes en producción con una base limpia?"*

Para lograr esto de forma quirúrgica sin perder configuraciones base ni romper el sistema, el Administrador DevOps ejecuta un script controlado de truncamiento que discrimina las capas de información:

Estructura del Script de Purga Quirúrgica (SQL):

```
-- 1. DESACTIVAR POLÍTICAS DE INTEGRIDAD TEMPORALMENTE
SET session_replication_role = 'replica';

-- 2. TRUNCAR EXCLUSIVAMENTE TABLAS OPERACIONALES / TRANSACCIONALES (DATOS DE PRUEBA)
TRUNCATE TABLE
    ope_facturas_sifen,
    ope_ordenes_trabajo,
    ope_inspecciones_dvi,
    ope_movimientos_inventario,
    ope_turnos_crm
RESTART IDENTITY CASCADE;

-- 3. NOTA: NO SE TOCAN LAS TABLAS DE CONFIGURACIÓN NI CATÁLOGOS BASE:
-- cat_repuestos (Mantiene el inventario real cargado)
-- cat_sucursales_bahias (Mantiene las rampas del taller configuradas)
-- auth.users / public.perfiles_empleados (Mantiene a los mecánicos y cajeros listos para
loguearse)

-- 4. REACTIVAR INTEGRIDAD REFERENCIAL
SET session_replication_role = 'origin';
```

Este procedimiento garantiza que el fin de semana previo al inicio oficial del sistema, el historial sucio de la capacitación desaparezca, dejando las secuencias numéricas (IDs de facturas y OTs) reiniciadas en 1, pero preservando intactas las cientos de horas invertidas por el personal cargando catálogos de repuestos, temporarios de servicios y configuraciones de red.

6. Resumen del Algoritmo de Seguridad para Mantenimientos

Para mitigar cualquier factor de error humano, el pipeline automatizado ejecuta el siguiente esquema cada vez que se detecta un cambio en el código fuente de la aplicación:

```
[Código Fuente Modificado] → [Push a Git] → [Pipeline Genera Backup Dump
Automático de la BD Viva]
↓
[Validación de Sintaxis SQL con Supabase CLI Lint]
↓
[Ejecución de "supabase db push" (Aplica solo deltas/cambios agregados)]
↓
```

[Despliegue del Nuevo Backend] → **[ÉXITO: Sistema Actualizado sin Pérdida de Datos]**